

System Design

1. What this is, in short

This describes a memory system for a conversational assistant.

It decides what is worth remembering. It stores it properly. It finds the right memories when a question arrives, and fits them into a limited space. And it manages their whole life — settling contradictions, letting old memories fade, and deleting them when asked — while keeping every user's data strictly separate and keeping secrets out entirely.

None of this was invented out of thin air. **Every component here answers either a requirement I worked out in Deliverable 1, or a failure I actually measured in Deliverable 3.** The research choices come from the scan in Deliverable 2.

The scope is a first version that one person can realistically build and run. Correct behaviour and safe handling of data come first. Enormous scale comes later.

2. The problem this solves

An assistant with no memory makes the user pay the same cost on every single conversation: explaining again who they are, what they are working on, and how they like to be helped.

So the system exists to do exactly one thing — **stop losing information between conversations.**

And one rule follows immediately from that. Memory is not storage. A database keeps everything. A memory system keeps only what will improve a future answer. So the first thing standing in front of the store is not a write — it is a decision about whether to write at all.

3. Who it is for, and what it is not

Who uses it. People talking to an assistant across many sessions. Two I have in mind specifically: a developer on a long project (my own .NET-to-Java migration), and a user of a product with many customers (CommentHook).

What it does. Remembers lasting facts and preferences. Recalls the right ones at the right moment. Honours a preference when the user changes it. Forgets when asked. Never mixes users up. Never keeps secrets.

What it deliberately does not do, yet. It is not a document search system or a general knowledge base. It does not handle images or audio. It does not learn what to forget. And it does not reason deeply about how facts change over time.

Each of those is a *deliberate postponement with a named trigger*, written down in the decision records — not something I forgot.

4. The limits it must live inside

What goes in: things the user says (to be remembered), and questions (to be answered). Both always tied to one logged-in user. **What comes out:** a ranked set of memories inside a budget — or an honest "I have nothing useful for this."

Limit	The target
How much context we may use	Never more than the caller allowed
Speed	Lookups finish in under 100 milliseconds, even in the slowest cases
Cost	Bounded per write and per lookup. The store itself is bounded by forgetting.
Privacy	No secrets saved. Deletion completes within 60 seconds.
Being right	The current preference beats the old one. Say "I don't know" rather than guess.
Keeping users apart	Zero results from another user. Ever.

Success is defined as beating the naive system on exactly the same test data. Relevant results from 0.20 up to above 0.6. The old preference winning from one time in three down to roughly never. Leaking another user's memory from 92% of the time down to never. Secrets findable from every one down to none. And guessing, from always, down to an honest silence.

5. The seven rules that must never be broken

These hold on every single request. Each one closes a failure I measured with real numbers.

Rule	What it says	The failure it closes
1	No user ever sees another user's memory — and the database itself enforces this	Leaking, measured at 92% of questions
2	Secrets are never saved	Every secret was stored and findable
3	We never use more of the context budget than we were given	Wasted budget
4	A deleted memory is gone from storage and search within 60 seconds	Deletion that does not really delete
5	The current preference always beats the outdated one	The old preference winning one time in three
6	If nothing is relevant, say so instead of guessing	Guessing 100% of the time
7	If the memory store is down, the assistant still answers without memory	Reliability

6. The pieces, and what each one does

See architecture.pdf for the picture.

The piece	What it does
Pulling facts out	Turns what the user said into short, clean facts and names
The write gate	Decides whether it is worth keeping, how important it is, and what type. The secrets filter runs before it.
Storage	The memories themselves, plus three different ways of searching them, all in one database
Search and ranking	Searches three ways, combines the scores, ranks them, settles contradictions, and decides whether anything is actually good enough
Building the context	Fits the winners into the budget, and hands them over as facts, never as instructions
The life of a memory	Fades, archives, summarises, and deletes
Safety (everywhere)	Keeps users apart, blocks secrets, logs everything, and really deletes
Monitoring (everywhere)	Measures whether memory is helping, and what it costs

7. How the data is organised

Full detail in data_model.md.

One table holds the memories. Each row records which user it belongs to, its type (an event, a fact, a preference, or the live chat), the content, two ways of searching it, how important it is, how confident we are, where it came from, when it was created and last used, how alive it still is, and a pointer to whatever replaced it.

The type drives the rules. Preferences get *replaced* when they change — and that is the structural fix for the old preference winning. Knowing where a memory came from, and how confident we are, is what lets the system answer "why do you think that?"

8. How information moves

See data_flow.pdf for the picture.

When the user says something: the message arrives, secrets are blocked immediately, the call returns straight away — and then, in the background, the facts get pulled out, the gate decides, contradictions are settled, and it is saved.

Blocking secrets happens immediately on purpose. That way a password is never written down anywhere, not even in a queue while it waits its turn.

When a question arrives: search this user's memories three ways, combine and rank, check whether anything is genuinely relevant (and admit it if not), fit the winners into the budget, and hand them over.

Overnight: work out how alive each memory still is, archive the ones that have faded, summarise old events together, and finish off any deletions.

9. Finding the right memories

The search runs three ways at once — by meaning, by exact keyword, and by matching names — all limited to a single user. The three scores are put on a common scale and combined:

score = 0.4 × how well it matches + 0.2 × how recent + 0.2 × how important + 0.2 × how often it is used , then adjusted by how confident we are.

When two memories disagree, the newer and more confident one wins, and the loser keeps a pointer to what replaced it. And if nothing clears the relevance bar, the system says so rather than guessing — the direct fix for a naive system that guessed every single time.

Then the winners are packed into the budget until it is full, never going over. They are handed to the model clearly labelled as *facts* — so that a memory containing "ignore all previous instructions" cannot hijack the assistant.

Those weights are a starting guess, to be tuned against a real benchmark. A second, more careful re-scoring model sits behind a switch, waiting on an experiment that will tell me whether its accuracy is worth the delay.

10. The calls other systems can make

Full detail in api_contracts.md.

Four things: send in a message, ask for the relevant memories, report that a memory is wrong, and delete one. The user is always worked out from their login token, never from the message body — so nobody can simply ask for someone else's data.

The most important error is the one for when memory is unavailable. Its contract says: carry on and answer the user anyway, without memory. Degrading gracefully is a designed-in behaviour, not an accident.

11. Privacy and security

Full detail in threat_model.md.

Users are kept apart by the database itself, so one forgotten filter in one query cannot leak anything. That is the direct structural answer to leaking 92% of the time. And secrets are blocked before they can ever be written down.

The threat model covers reading another user's data, keeping secrets, poisoning the memory with false facts, hidden instructions planted inside a memory, deletion that does not really delete, fishing for other people's data, and impersonating another user. Each has a specific defence.

The cross-user attack test and the deletion test are the non-negotiable gates carried into Deliverable 6.

12. Monitoring and testing

While it runs: how often memory is used, how often users correct it, how fast it is, and what it costs. Those four numbers tell me whether memory is helping at all. This habit comes straight from watching the bank's fraud models every week.

Offline: the benchmark adopted in Deliverable 2, whose question categories line up with my failures. Plus a re-run of the whole Deliverable 3 test, so every claim can be compared directly against the naive baseline.

13. Scale, speed, and recovering from failure

Scale. Thousands to a few million memories per user, on one database. Deliverable 3 showed that even scanning everything took under a millisecond at 2,000 memories — so a specialist search database only earns its cost well beyond that. That is a measured decision, not a guess.

Speed. Lookups under 100 milliseconds, even in the slowest cases. The slow work of pulling facts out happens in the background, where nobody is waiting.

Recovering. If the store goes down, the assistant answers without memory. The background jobs can be safely re-run if they fail halfway. And deletion happens in two steps, so even a crash mid-way still leaves the memory invisible to every read.

14. The decisions I made

Full reasoning in `decision_records/`.

Decision	What I chose
ADR-001	Start the write gate as an AI judge; move to a trained classifier once volume justifies it
ADR-002	Use hybrid search now; postpone the two-date graph behind a named trigger
ADR-003	Use one PostgreSQL database instead of three specialist ones
ADR-004	Let the database keep users apart, rather than trusting application code to remember
ADR-005	Check for secrets immediately; do everything else in the background
ADR-006	Keep the re-scoring model behind a switch until I have measured what it costs in speed
ADR-007	Use a simple forgetting formula; postpone the machine-learned version

15. Milestones and gates

Full plan in `sprint_plan.md`.

Seven milestones, ordered so each one closes a specific failure I measured, and each one carrying a real number it has to beat on the same test data.

The non-negotiable gates: leaking another user's memory, from 92% of questions down to zero. Secrets findable, from every one down to none. The old preference winning, from one time in three down to roughly never. Deletion completing within 60 seconds. And a full comparison against the naive baseline.

A milestone that misses its gate is not finished — no matter how good the demo looks.

16. Risks and open questions

- Risks I am left with.** Secret detection is never perfect — reduced by encryption, logging, and a feedback loop. And an AI-driven write gate can be manipulated by carefully crafted text — reduced once it becomes a trained classifier.
- Postponed, each with a trigger.** The two-date graph (if my simpler rule fails the benchmark questions about facts that change). The re-scoring model (if it clears the speed bar). Machine-learned forgetting (if tuning the simple formula becomes a genuine bottleneck).
- Still open.** How much evidence should promote a passing remark into a lasting memory? How do I tune the ranking weights honestly, without a labelled set of correct answers? And do a small product and a regulated bank actually need different approaches to keeping users apart?

What I can claim, and what I cannot. Every component here traces back to a requirement, a rule, or a failure I measured. The details live in the companion files, so another engineer could build this without ever needing to talk to me.

What this design *cannot* yet claim is that it works. That is Deliverable 6 — where every gate above gets tested against the very baseline it was built to beat.